

PART I - Computer Science 1 (CS1)

Chapter 1

Common CS1 Activities and Lessons

April Crockett¹, Srishti Srivastava²

¹Tennessee Technological University, acrockett@tntech.edu

²University of Southern Indiana, fsrishti@usi.edu

Abstract

This chapter distills some common CS1 activities, implementation strategies, challenges, and lessons learned across the eight institutional CS1 exemplars in this volume. It serves as a shared resource chapter between the project overview in Chapter 0 and the institution-specific CS1 chapters 2-9 that follow. Rather than presenting another local course implementation, the chapter identifies recurring activity families and reusable guidance that cut across TNTECH, Knox, USI, UNL, Webster, Casper, HPU, and MSU. Its purpose is to help instructors locate activities that can be adopted as a single class activity, a short module, a laboratory exercise, or part of a broader course-level infusion of parallel and distributed computing (PDC) into CS1, all the while ensuring that the basic CS1 learning outcomes are preserved.

The activities summarized here introduce accessible versions of core PDC ideas, including data parallelism, task decomposition, speedup, load balance, contention, communication, distributed data access, APIs, event handling, asynchrony, and limits of parallelism. These ideas are connected to familiar CS1 topics such as loops, arrays, searching, sorting, functions, objects, graphical interaction, data access, and basic algorithmic reasoning. Recurring activity families include unplugged activities such as Flag Maker and Penny Search/Penny Sorting; visualizations and simulations for parallel search, linked lists, flag coloring, and Zombie Attack; and lightweight programming activities involving remote earthquake-data APIs, Greenfoot or event-driven programming, and parallel search or sorting.

The chapter points readers to institutional chapters and their repositories, appendices, assessment instruments, student feedback, and classroom evidence so they can select and adapt materials that match their own teaching context. The repositories would be curated periodically with updated instructional material and evidence.

Across institutions, shared lessons include starting with intuition, using unplugged or visual activities before code, keeping programming tasks lightweight, and embedding PDC concepts within existing CS1 learning goals rather than treating them as disconnected advanced content.

1.1 Introduction

Chapter 0 presents the overall motivation, project design, and roadmap for this volume. This chapter now turns from the project-level view to the shared CS1 activities and lessons that emerged across the institutional exemplars. Unlike Chapters 2–9, which describe local course implementations, this chapter is organized around reusable activity families. Its purpose is to help instructors quickly identify activities that may fit their own CS1 course and then move to the institutional chapters for nuanced implementation details, adaptations, evidence, and repository resources.

The chapter is intended for selective, adoption-oriented reading. An instructor may use it to find a single unplugged activity, a short visualization, a programming assignment, an assessment instrument, or a starting point for broader CS1 infusion. The activities summarized here are deliberately connected to familiar CS1 topics. Students may first act as processors, divide work, observe idle time, or predict the behavior of an animation before seeing how the same ideas connect to loops, arrays, functions, objects, APIs, graphical interfaces, or real-world data. This conceptual-to-practical progression helps introduce PDC without requiring students to become parallel programmers in their first course.

Table 1.1 provides a quick activity finder for this chapter. The first column briefly describes each activity and points to the section where readers should begin. The next two columns highlight the PDC ideas and the CS1 connections they reinforce. The last column cites which institutions have employed these and their estimates of classroom time utilized.

Table 1.1: Activity finder for CS1 PDC activities described in this common chapter.

Activity and brief description	Primary PDC ideas	CS1 anchors	Duration / where used
Unplugged, visual, and simulation activities			
Flag Maker. Students act as processors coloring flag-grid cells in sequential and parallel scenarios. § 1.3.1.	Data parallelism; task decomposition; speedup; contention; pipelining; critical path; race conditions.	Loops; grids; repeated operations; graphical output; independent work and dependencies.	40–60 minutes to one 75-minute class. See USI § 4.3.1 Table 4.9, HPU § 8.3.1 Table 8.3, TNTECH § 2.3.2, and MSU Flag Maker section.
Penny Search / Penny Sorting. Students search or sort pennies under sequential, balanced-parallel, and load-imbalanced scenarios. § 1.3.2	Data parallelism; load balance/imbalance; slowest worker; communication overhead; Amdahl’s Law; observed versus ideal speedup.	Searching/sorting; timing; comparing strategies; reasoning from measured performance.	Fits a 50- or 75-minute class. TNTECH: 30–40 minutes of a 75-minute lab § 2.3.1; USI: 75 minutes § 4.3.2 Table 4.10; HPU: 45–65 minutes § 8.3.2 Table 8.6; Casper: § 7.3.2.
Animations and simulations. Students use visualizations of parallel search, linked-list concurrency, flag coloring, and Zombie Attack behavior. § 1.3.3.	Work partitioning; speedup limits; synchronization; race conditions; mutual exclusion; dependencies; domain decomposition.	Arrays/lists; linear search; linked-list operations where covered; grids; prediction before code.	Webster animations are 10–15 minutes each; Zombie Attack may introduce a longer assignment. Add/verify Webster labels before formal references.

Continued on next page.

Table 1.1 continued.

Activity and brief description	Primary PDC ideas	CS1 anchors	Duration / where used
Plugged-in programming activities			
Programming Flag Maker. Students draw simplified flags on a 2D grid using nested loops and graphical output. § 1.4.1.	Independent drawing operations; dependency reasoning; data-parallel structure; possible grid decomposition.	Nested loops; row/column indexing; 2-D grids; graphical output; testing.	Knox: Week 3 lab/homework and dependency-graph discussion § 3.3.1. USI: 75-minute launch plus one week paired programming § 4.4.1 Table 4.11.
Greenfoot / event-driven work. Students build or modify graphical programs whose objects respond to events. § 1.4.2.	Event handling; asynchronous responses; active-object interaction; framework-controlled execution.	Objects; methods; classes; conditionals; graphical interaction; game/simulation logic.	Knox places Greenfoot in the event-driven/OOP portion of CS1, but duration/evidence are not yet stated . USI: optional 75-minute demo § 4.4.3 Table 4.13.
Earthquake Tracker / remote data. Students retrieve USGS data, parse JSON, filter events, and display selected information. § 1.4.3.	Distributed data access; client-server model; APIs; JSON data; retrieve–parse–filter–display workflow.	Functions; loops; arrays/collections; libraries; Makefiles; data processing; external input.	TNTECH: § 2.4.1; Casper: § 7.4.3; UNL: about 1.25 hours for USGS encapsulation lab (Table 5.3).
Assessment and evidence			
Course-wide surveys. Common pre/post items capture background and self-reported experience with programming and selected PDC concepts. § 1.5.	Parallel processing, distributed computing, event handling; pre/post comparison; adoption evidence.	Programming background; languages; data types; expressions; branching; loops; functions; arrays; classes/objects.	Usually administered near course start and end.

A useful reading strategy is to begin with the activity family that best matches the local adoption goal. An instructor seeking a single class activity may start with Flag Maker, Penny Search/Penny Sorting, or one of the animations. An instructor seeking a short lab or programming assignment may begin with Programming Flag Maker, Greenfoot, or Earthquake Tracker. An instructor planning broader CS1 infusion should read Section 1.2 together with the institutional chapters to compare how different teams created space for PDC, aligned activities with existing CS1 topics, and collected evidence.

Additional CS1 PDC activities not described in this chapter are listed in Table 1.2.

Table 1.2: Additional institutional CS1 PDC activities not described as full activities in this common chapter.

Activity and brief description	Primary PDC ideas	CS1 anchors	Duration / where used
Unplugged, visual, or simulation activities			
zyBooks parallel-computing section. Students complete an interactive textbook section as a low-barrier conceptual introduction to parallel computing.	Parallel-computing vocabulary; conceptual exposure; preparation for later activities.	Interactive textbook work; vocabulary; preparation for programming examples.	Casper: § 7.3.1. Expected time should be stated.

Continued on next page.

Table 1.2 continued.

Activity and brief description	Primary PDC ideas	CS1 anchors	Duration / where used
Coin Addition. Students collaboratively add coin values to illustrate shared work, shared state, and synchronization issues.	Shared state; race conditions; synchronization; coordination.	Variables; accumulation; updating shared totals; order of operations.	Casper: § 7.3.3. Duration/evidence should be clarified.
More Processors are Not Always the Best. Students examine why adding more processors does not automatically improve performance.	Speedup limits; overhead; diminishing returns; tradeoffs.	Runtime interpretation; comparing strategies; efficiency.	USI: § 4.6.1. Duration/material pointers should be verified.
MP Writing Activity. Students coordinate a writing task to experience communication, sequencing, and limits of parallel work.	Message passing; communication; coordination; limits of parallel work.	Instruction writing; sequencing; communication clarity.	HPU: § 8.6.1; Table 8.12.
Plugged-in activities and extensions			
OpenMP data-parallelism assignment. Students use OpenMP to experiment with data-parallel work and observe speedup and setup overhead.	Shared-memory parallelism; threads; data parallelism; speedup; scheduling/setup overhead.	Arrays; loops; compilation; command-line execution; runtime interpretation.	TNTECH: § 2.4.2; Figure 2.17 and related figures.
OpenMP array creation and array sum. Students use OpenMP in array-creation and summation tasks to explore work sharing and parallel execution.	Shared-memory parallelism; threads; work sharing; shared variables; reduction-style reasoning.	Arrays; loops; variables; summation; compiler/setup practice.	Casper: § 7.4.1, § 7.4.2. Duration/setup burden should be clarified.
Parallel Penny Sort programming activity. Students use a plugged-in version of penny sorting to compare sequential and parallel strategies.	Parallel decomposition; sequential/parallel comparison; speedup; workload division.	Sorting; arrays/collections; code tracing/review; performance interpretation.	USI: § 4.4.2; Table 4.12. HPU: § 8.4.1; Table 8.8.
Codeless PDC modules. Students complete low-setup modules that introduce PDC concepts without requiring full parallel-programming implementation.	Parallel/distributed models; decomposition; speedup; PDC vocabulary; low-setup exposure.	Module completion; conceptual reasoning; quizzes/short responses; asynchronous learning.	UNL: Table 5.3. Add section labels for module subsections if cross-references are needed.
Zombie Attack programming assignment. Students extend a grid-based simulation in which neighboring regions interact as infection spreads.	Domain decomposition; boundary interaction; state updates; neighboring-region communication.	2-D grids; simulation loops; conditionals; object/state updates.	Webster Zombie Attack Assignment section. Add/verify section label before formal reference.
Knoxcraft. Students work in an interactive, game-like programming setting that connects objects, events, and framework-mediated behavior.	Event-driven interaction; object behavior; possible framework-mediated execution.	Objects; methods; game-style interaction; program structure.	Knox activity section. Duration, evidence, and material pointers remain underdeveloped.
Physical computing projects. Students use Raspberry Pi/GPIO projects involving traffic lights, buttons, multiple processes, and a temperature sensor.	Multiple processes; event-driven behavior; hardware interaction; concurrency-related reasoning.	Python; functions; hardware I/O; sensors/buttons; debugging.	Casper: § 7.4.4, § 7.4.5, § 7.4.6. Frame as optional/experimental due to setup burden.
Client-server Battleship idea. Students explore a client-server game design as an extension involving distributed state and communication.	Client-server interaction; distributed state; communication protocol; networked application structure.	Objects/structs; functions; design; input/output; debugging multi-part programs.	UNL Other Activities and Ideas: § 5.6. Clarify whether adoption-ready or future extension.

The remainder of the chapter follows this adopter-facing structure. Section 1.2 summarizes how CS1 was changed across institutions and the challenges that arose. Sections 1.3.1, 1.3.2, and 1.3.3 describe unplugged, visual, and simulation-based activities. Sections 1.4.1, 1.4.2, and 1.4.3 describe plugged-in activities. Section 1.5 summarizes the common course-wide pre/post survey structure. The conclusion distills cross-institution lessons for instructors who wish to adopt, adapt, or extend these materials.

1.2 How CS1 was changed and PDC topics integrated (April)

Across the participating institutions, parallel and distributed computing topics were incorporated into CS1 through three broad, and sometimes overlapping, strategies: selectively reducing existing content, modifying current assignments and activities, and adding short PDC-focused modules to the established course [2]. The institutions represented a wide range of instructional settings, including public and private universities, small liberal arts colleges, and larger research institutions. Their courses also differed in programming lan-

guage, class size, contact hours, laboratory structure, and semester length. Because of these differences, the project did not prescribe a single model for changing CS1. Instead, each institution adapted the materials to fit its local curriculum while preserving the foundational programming knowledge expected of beginning students.

Some institutions created space by reducing or deferring topics considered specialized, redundant, or better suited for CS2. Examples included binary and random-access files, C-style strings and arrays, detailed output formatting, two-dimensional arrays, advanced class relationships, static members, garbage collection, or additional practice with object-oriented concepts. In most cases, these topics were not viewed as unimportant; rather, instructors determined that they could be covered more briefly or revisited in later courses. Other institutions retained the full CS1 syllabus and introduced PDC without removing content. One institution, for example, placed three asynchronous PDC modules outside regular class time, while another used existing laboratory and recitation periods to provide PDC activities without restructuring the lecture sequence. Across institutions, the amount of direct instructional time devoted to PDC ranged from no scheduled class time for asynchronous modules to approximately seven hours, generally representing one to two weeks of student engagement.

The most common strategy was to embed PDC concepts into topics already taught in CS1. Existing instruction on loops, arrays, searching, file processing, classes, objects, and problem decomposition was reframed to include ideas such as data parallelism, task decomposition, distributed data access, event handling, scalability, speedup, and computational cost. Examples included modifying array summation and searching assignments to discuss parallel execution, using serial and parallel versions of programs so students could compare execution times, accessing earthquake data through remote APIs, introducing OpenMP within an arrays laboratory, and using event-driven game programming to teach classes and objects. At other institutions, Flag Making 1.3.1, Penny Search 1.3.2, Minecraft, visualizations, animations 1.3.3, and other exercises were used to help students reason about parallelism before encountering implementation details. These changes generally layered PDC learning objectives onto traditional CS1 objectives rather than replacing them. Students still practiced loops, arrays, classes, file processing, and algorithmic thinking, but did so within contexts that reflected contemporary multicore, distributed, and event-driven computing.

The form of PDC instruction also varied according to institutional needs and instructor expertise. Some courses emphasized conceptual understanding through readings, discussions, animations, and codeless demonstrations. Others included guided programming exercises using OpenMP 2.4.2, Java parallel libraries, remote data sources, or event-driven environments such as Greenfoot 1.4.2. Several institutions used unplugged activities in which students physically modeled processors, divided work, or compared serial and parallel approaches. This variety was intentional: the goal was not to make students proficient parallel programmers during their first course, but to establish an early awareness that computation is not always sequential and that modern problems may involve multiple processors, distributed data, asynchronous events, and performance tradeoffs. Taken together, the implementations demonstrate that PDC can be introduced through incremental and locally appropriate changes rather than through a complete redesign of CS1.

1.2.1 Challenges (April)

The most significant challenge was *finding instructional space in an already crowded CS1 curriculum*. Nearly every existing topic could be justified as useful, making decisions about what to condense difficult. The redesign therefore required instructors to distinguish between concepts that were essential for immediate student success and topics that were specialized, repeated in later courses, or could be introduced with less detail. Even when no topic was eliminated completely, reducing practice or lecture time carried the risk that students would receive less reinforcement of traditional material. Careful alignment among course outcomes, assignments, laboratories, and later courses was necessary to ensure that the revised curriculum did not create gaps in students' preparation.

A second challenge involved *presenting PDC at an appropriate level for novice programmers*. Students in CS1 are simultaneously learning syntax, control structures, debugging, and problem-solving strategies. Introducing threads, race conditions, synchronization, or implementation-level details too early could create unnecessary cognitive load. Consequently, the revised course emphasized conceptual understanding, visual demonstrations, bounded code examples, and unplugged activities before asking students to interact with parallel code. The intent was not for students to become proficient parallel programmers in CS1, but for them to recognize that computational work can be divided, understand that modern computers contain multiple processing units, and begin reasoning about the potential benefits and limitations of parallel execution.

The redesign also required substantial *faculty effort*. Developing age-appropriate examples, identifying suitable locations in the existing curriculum, revising assignments, testing software and datasets, and repeatedly refining the materials required more than 100 hours across the initial development year. Coordination with laboratory instructors and teaching assistants added another layer of complexity because those supporting the course needed enough familiarity with the new concepts and technologies to guide students effectively. Technical differences among student computers, compiler configurations, OpenMP support, and execution timing also had to be anticipated. Although adopting already-developed modular materials can substantially reduce this workload, successful implementation still requires local adaptation, instructor preparation, and continued refinement based on student responses.

1.3 New Unplugged Activities

1.3.1 Flag Maker (David, April)

Activity Summary

The Flag Maker unplugged activity was the most widely adopted activity created by our project. It has students take on the role of the processor to color cells of a grid in order to make the image of a flag. They do this sequentially and then in a variety of parallel configurations. The activity illustrates speedup and several concepts of coordination and contention in a graphical manner. It is a parallelized version of a graphical but non-parallel programming assignment to teach about loops by Emad [4].

This activity has been previously published [9]; the material on it here draws from that document.

Prerequisites

The prerequisites for the Flag Maker activity are minimal. It has been used successfully as early as 1/3rd of the way into CS1, but should be suitable even for a CS0 class. The students need to understand the idea of operations executing one at a time and how loops allow repetition, but they do not even need conditionals or functions. The activity is built on the idea of the computer being able to perform multiple simultaneous operations, but we found that we could just tie this to the terms “dual-core” and “quad-core”, with which the students were already familiar.

Learning Outcomes

Upon completing the activity, participants will be able to:

1. recognize parallel computing terms such as speedup, contention, and pipelining;
2. recognize algorithmic solutions that are representative of data parallelism;
3. describe how parallel execution time is bounded by the slowest processor (i.e., the critical path).

Logistics

We split the class into groups of 5–6 or groups of 2–3, pairs of which merged to create groups of 5–6 after the first two phases of the activity. Each group needs 4 sheets of paper (1 per phase) with a large grid, each cell of which will be a single pixel in their drawing. (Information on the paper to use is in Section 1.3.1.) They also need one drawing implement of each of the flag’s colors (red, blue, yellow, and green). The most popular implement is thick markers, but thin markers, bingo daubers, colored pencils, or crayons can also be used; expect unhappy students if they are provided with pencils or crayons, though giving these to some groups is a pedagogic choice discussed below. Each group will also need a way to keep time; we had students use the stopwatch program on their phones.

Running the Activity

Initially, the instructor introduces the activity by telling students they will take the role of computer processors to color a flag. Introduce the flag of Mauritius (see Figure 1.1), an island nation off the east coast of Africa. Split the class into appropriate groups and pass out the materials. It is also helpful to clarify the expectations on coloring: each person should color a single grid cell at a time (not scribble across a whole row) and to fill in the cells consistently. Consistency is the main requirement here, but we tend to encourage a style where each cell is colored pretty thoroughly, but not perfectly. (Perfection will greatly slow down the activity, but you want something that looks like the flag when completed.) It is helpful to show the class a sample flag that you’ve already colored.



Figure 1.1: Flag of Mauritius [12]

Then, the activity proceeds in four scenarios, each with a different configuration of students coloring the flag. These scenarios are depicted in Figure 1.2, where P1–P4 mark the cells to be colored by up to four “processors” in the scenario. In the first scenario, one student (P1) in each group colors all the grid cells, one color at a time, while another student times them. After all the teams are done, have them report the times and record them on the board.

In Scenario 2 (top right), the flag is split in half, with one student (P1) coloring the red and blue stripes while another student (P2) colors the yellow and green stripes. In addition, there is a third student timing them; the time should be the finishing time of whichever student finishes last. After the coloring is completed, the times should again be recorded on the board.

Scenario 3 (bottom left) and Scenario 4 (bottom right) split the flag into 4 parts, each colored by one student, while a fifth student times them. Again, the time should be the finishing time of the last processor and the times should be collected on the board when all the groups have finished. In Scenario 3, each student colors one entire stripe. In Scenario 4, each student colors a vertical strip which includes two columns of each stripe. The cell numbering is also important here; each student is expected to color their first column, coloring red, blue, yellow, and green before returning to the top of their second column to repeat the colors. Note that each group should still have only one drawing implement of each color so the implements will have to be passed around.

Activity Discussion

Some institutions choose to discuss each scenario directly after each one, while others choose to have class discussion after the activity has ended. In both cases, the purpose of the discussion is to ask students what they observed and to put their observations into the context of parallel computing giving official terminology.

The first observation is likely that the coloring times decreased with the number of people doing the coloring. This leads to the definition of speedup (serial time divided by parallel time) and its calculation for each scenario. A particular term of interest is linear speedup, when the speedup is equal to the number of cores. More common is sublinear speedup, which occurs because the parallel program has to do extra work (like system calls to use

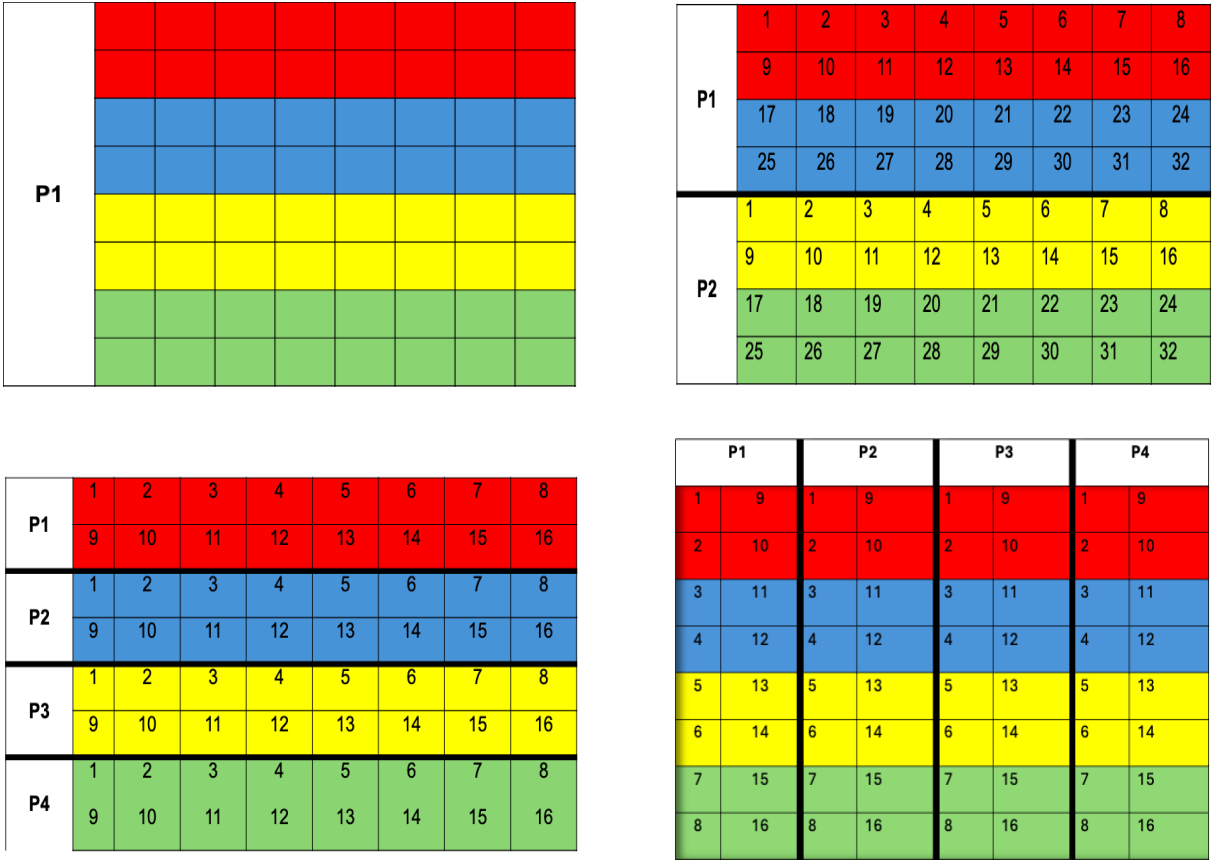


Figure 1.2: Graphical depiction of the Flag Maker scenarios: Scenario 1 (top left), Scenario 2 (top right), Scenario 3 (bottom left), and Scenario 4 (bottom right). P1–P4 depict the regions to be colored by each “processor”. The cells after Scenario 1 are numbered to show the order in which they should be colored.

all the cores) or some of them have to spend time waiting for a limited resource (like the markers in the activity). In parallel computing, extra work is overhead and resource-induced delays are contention. It is also possible to achieve superlinear speedup, one greater than the number of cores. In the activity, this may occur because students get more efficient with coloring as they practice or color less thoroughly as they get tired. Getting more efficient is somewhat like “warming up” a system, where a second run of the program can benefit from caching effects. In real systems, superlinear speedup is rare but can occur when performance is limited by memory rather than processing; running on multiple

Instructional Material

- TNTECH CS1 Flagmaker Presentation Slides
- TNTECH CS1 - Grids to Print for Activity
- <https://www.youtube.com/watch?v=WmW6SD-EHVY>, a video demonstrating the potential speedup of coloring all pixels simultaneously instead of one at a time. This is framed as the difference between a CPU and GPU. We suggest it as something the students could be asked to watch before the activity.

Example Implementations

The following institutions are using a variation of this assignment and you can find their details in their specific chapters:

- Knox CS1 - Section 3.3.1; Knox does post-activity discussion and has a Flag Maker plugged activity (discussed in Section 1.4.1).
- TNTECH CS1 - Section 2.3.2; TNTECH does discussion after each Scenario and has an OpenMP programming assignment 2.4.2 after the activity concludes.
- Webster CS1 - Section ??

Experience and Teaching Tips

Data and Analysis

1.3.2 Unplugged Penny Search Activity (Srishti)

Activity Summary

The Unplugged Penny Sorting Search Exercise (PE) is an unplugged, hands-on active learning activity designed to introduce foundational concepts in parallel computing through a tangible, accessible task. Rather than relying on code or simulation, participants physically sort and search a collection of pennies to find the oldest one, enacting the roles of processors in a parallel system. The activity unfolds across three phases that progressively model a uniprocessor baseline, a balanced two-processor parallel execution, and a load-imbalanced five-processor scenario. By physically experiencing the difference in task completion time

across these phases, participants develop an intuitive understanding of how parallel systems behave under varying conditions of data distribution and workload balance. Below is a more detailed description of the activity phases:

Phase 1: Uniprocessor phase At the data partitioning step, each group will receive a bag of pennies (e.g., 50), though the number can be reduced as long as the count remains an even number. In each group, Participant 1 performs sorting and searching for the oldest penny as their task on the allocated data (pennies), Participant 2 records time as their task and the other participants (if any) perform the task of observing. At a set time, all groups and group members begin their respective tasks.

- Participant 1 Task: find the oldest penny
- Participant 2 Task: tracks the time Participant 1 takes to complete the task and documents the results.
- Other participants: observe Participants 1 and 2. When they are done, shuffle coins and put them back in the bag.

Phase 2: Two Processor Parallel Phase At a set time, all groups will begin the tasks.

- Participants 1 and 2 of each group are given an equal number of pennies (e.g., 25). This workload or data distribution is managed by the instructor to create a load balanced parallel execution scenario.
- Participant 1 Task: Find the oldest penny in their bag.
- Participant 2 Task: Find the oldest penny in their bag.
- Participants 1 and 2 communicate and determine the overall oldest penny between both bags.
- Participant 3 Task: tracks the overall time for Participants 1 and 2 to determine the oldest penny in both bags.
- Any other participants: observe Participants 1, 2, and 3. When they are done, shuffle coins and put them back in the bag.

Parallel time is calculated as the time of the slowest participant among 1 and 2 and the communication time to determine the oldest penny of the two participants. When done observing, participants in the group shuffle coins and put them back in their respective bags. Each bag should have an equal number of pennies.

Phase 3: Five Processor Parallel Phase The activity facilitator or instructor gives each group four bags of two pennies for participants 1, 2, 3, and 4 and the remaining pennies (e.g., 42) to Participant 5. This uneven workload or data distribution introduces a load imbalance scenario for the parallel processes. At a set time, sorting begins for all groups.

- Participants 1-5 Task: Each must find the oldest penny in their bags. Once all participants complete their sorting, they must communicate with each other to identify the overall oldest penny.

- Participant 6 Task: tracks the overall time for Participants 1-5 to determine the oldest penny in their bags.
- Any other participants: observe Participants 1, 2, 3, 4, 5, and 6. When they are done, shuffle coins and put them back in the bag.

Parallel time is calculated as the time of the slowest participant among participants 1-5 and the communication time to determine the oldest penny of the five participants.

Prerequisites

This activity requires no formal prerequisites in parallel computing, making it suitable for introductory CS courses (CS1 and CS2) as well as upper-division systems courses where the instructor wishes to ground abstract concepts in concrete experience. Familiarity with basic algorithmic thinking, such as the idea that a search task has a measurable runtime, is sufficient preparation. The activity is also accessible to participants from non-CS disciplines participating in computing outreach or interdisciplinary courses.

Learning Outcomes

Upon completing the activity, participants will be able to:

1. explain the concept of data parallelism and how a dataset can be partitioned across multiple processors;
2. distinguish between load-balanced and load-imbalanced parallel workloads and articulate the performance implications of each;
3. describe how parallel execution time is bounded by the slowest processor (i.e., the critical path);
4. explain the concept of Amdahl's Law;
5. recognize the role of inter-processor communication in determining overall parallel performance; and
6. compare theoretical speedup with observed speedup from recorded timing data across the three phases.

Implementation Details

The PE activity is delivered as part of a self-contained instructional module on parallel computing. The module follows a deliberate pedagogical arc: it begins by motivating the relevance of PDC as a curriculum topic, transitions into conceptual grounding through short video resources, moves into the unplugged activity itself across three phases, and closes with structured group discussion and an optional survey. This arc is consistent whether the module is taught as a standalone guest lecture, a lab session, or an integrated class period within a broader CS1 or CS2 course.

The overall session outline is as follows: (1) motivation for why PDC belongs in the undergraduate curriculum, (2) an introduction to the concept of parallel computing through curated video resources, (3) the unplugged penny sorting activity across its sequential and parallel phases, (4) a group and class-wide discussion reflecting on observed results, and (5) an optional post-activity survey to capture student perceptions. Each component is described in detail below.

(i) Instructional Material (Lectures, Videos, Assignments, Tests, Discussions)

- **Motivation:** the session opens with a brief framing of why PDC is a required component of the undergraduate computing curriculum, grounded in the joint initiative mandated by the National Science Foundation (NSF) and the ACM/IEEE Computer Society. Pointing students to the PDC curriculum rationale (available at <https://grid.cs.gsu.edu/~tcpp/curriculum/>) situates the activity within a broader disciplinary context and helps students understand that parallel thinking is not an elective skill but a core competency expected of modern computing graduates. This framing takes no more than three to five minutes and significantly increases buy-in, particularly in courses where students may not have expected to encounter any PDC content.
- **Video resources:** before launching into the physical activity, a short video segment introduces the concept of parallel computing at an accessible level. Two videos are recommended for in-class use: a light, metaphor-driven introduction ("Digging Holes," available on YouTube) and a more substantive technical overview (available on YouTube). The LLNL parallel computing tutorial available ([here](#)) and Chris Bourke's webpage at University of Nebraska at Lincoln are recommended as post-class resources for students who wish to explore further. The two in-class videos together run approximately eight to twelve minutes and can be trimmed depending on available time. Their purpose is not to teach parallel computing comprehensively but to give students just enough conceptual vocabulary (processor, task, speedup, parallelism) to make sense of what they are about to enact physically.
- **Activity materials:** the primary instructional artifact is the activity itself, which requires minimal preparation overhead. One bag of pennies per group (approximately 50 pennies, though smaller or larger counts work if the count is even) is needed for Phases 1 and 3. For Phase 2, two bags per group containing an equal split (e.g., 25 pennies each) are required. Each group also needs a timer, and it is recommended that the instructor prepare an optional recording sheet on which groups log the elapsed time for each phase, the oldest penny identified, and any observations about coordination or idle time. This sheet doubles as a data collection instrument for the post-activity discussion. Note: if acquiring pennies is challenging, then instructors may also use custom index cards with random years printed on them, craft buttons with years printed on them using a permanent marker, or playing cards. April Crockett's TNTECH instructor presentation slides that walk through the scenarios and printable lab sheets are available on GitHub.

- **Assignments, tests, assessments:** To reinforce the concepts introduced through the activity, a short follow-up assignment or test can ask students to calculate the observed speedup from Phase 2 and Phase 3 using their recorded timing data, compare it against ideal (linear) speedup, and reason about the sources of deviation. Students can further be asked to connect their observations to Amdahl’s Law by estimating the parallelizable fraction of the task and quantifying the overhead introduced by inter-processor communication. An optional post-activity survey (such as the ASPECT survey [14]) is recommended for capturing student perceptions of the activity’s clarity, engagement, and perceived learning value. Survey data is particularly useful for instructors iterating on the activity across semesters or presenting the pedagogy at teaching focused venues.

(ii) How-To Guide

- Session structure: plan for at least a 50-minute class period. A suggested time allocation is: five minutes for PDC motivation, ten to twelve minutes for video viewing, twenty to twenty-five minutes for all three phases of the activity (including penny redistribution between phases), and ten to fifteen minutes for the group and class-wide discussion. If time is limited, the motivation segment and one of the two videos can be reduced without compromising the activity itself.
- Before class: prepare one bag of approximately 50 pennies per group for Phases 1 and 3, ensuring each collection spans a wide range of years so that the search is non-trivial. Prepare two additional bags per group with equal penny counts for Phase 2 (e.g., 25 each). Print or otherwise distribute the optional recording sheet. Confirm that each group has access to a timer. If using the videos in class, test the links and audio in advance.
- Forming groups and running phases: divide the class into groups of five or six. Groups of four can participate in Phases 1 and 2 with minor adjustments but require at least five participants to enact Phase 3’s load imbalance scenario meaningfully, since the contrast between participants with two pennies and the participant with the bulk of the coins is the core pedagogical mechanism of that phase. Run Phases 1, 2, and 3 according to the description provided above.
- Group and class discussion: after all phases conclude, facilitate a discussion at the group level first, then open it to the full class. Prompt students to share their recorded times, compare across groups, and articulate why Phase 3 did not improve on Phase 2 despite engaging more participants. Key observations to draw out include: the parallel runtime is bounded by the slowest participant; Participants 1 through 4 in Phase 3 were idle for most of the phase; and the act of comparing results across participants introduces communication overhead that is non-trivial even at small problem sizes. This is also a natural point to introduce or revisit Amdahl’s Law formally.
- Assessments: conclude the session with a follow up assignment, test, or a post activity survey.

Experience and Teaching Tips

This activity runs effectively in a standard 50-minute or a 75-minute class period, with approximately 20 to 25 minutes devoted to all three phases and the remainder reserved for the post-activity discussion and assessments. In practice, Phase 1 tends to take the longest for individual participants and provides a useful baseline that makes the speedup in Phase 2 viscerally apparent. The dramatic slowdown (or absence of speedup) in Phase 3 due to load imbalance consistently generates spontaneous reactions from students, which makes the subsequent discussion highly engaging. One common facilitation challenge is that participants in the load-imbalanced phase (Participants 1-4) finish their two-penny assignments almost instantly and then sit idle while Participant 5 continues sorting. This idle time is itself a powerful teaching moment as students directly experience what it means for processors to be underutilized. Instructors should call attention to this idleness explicitly, as it connects naturally to the concept of processor starvation and the importance of effective scheduling. Timing precision varies across groups, and recorded times may differ more than expected from the theoretical speedup. Rather than treating this as a flaw, instructors can use the variance to discuss real-world sources of overhead: participants search at different speeds, communication takes time, and the act of comparing two pennies is not instantaneous. This mirrors the behavior of real parallel systems and reinforces why the theoretical speedup is an upper bound rather than a guarantee.

For larger class sizes, it is advisable to have a teaching assistant circulate across groups during Phase 2 and Phase 3 to ensure that the penny distribution is carried out correctly and that timing starts simultaneously across all groups. For very large lectures, the activity can alternatively be demonstrated at the front of the room with one volunteer group while the rest of the class observes and predicts the outcome of each phase before the timer is stopped. Finally, instructors should allow participants to observe the timing data first and then derive the intuition themselves before the formal definition of Amdahl's law is introduced, as this leads to considerably stronger engagement and retention. The activity works best as a discovery experience, not an illustration of a concept already explained.

Data and Analysis

This activity was originally proposed by Mary Smith and Srishti Srivastava, with consultation from Brett A Becker [10]. In the initial phase of the development of this activity, its implementation was primarily evaluated through a student engagement survey based on the Wiggins et al. (2017) ASPECT survey[14]. the modified ASPECT survey, as presented in Figure 1.3.

This survey evaluates three key constructs: the perceived value of the activity, instructor contribution, and personal effort. The responses to each survey question were measured on a 5-point Likert scale, with values ranging from 1 (Strongly Disagree) to 5 (Strongly Agree). Faculty observational data were gathered during the students' participation in the activity.

The demographic variables, including gender, major, class standing, and age, were collected to explore possible relationships between these factors and engagement constructs. In summary, our analysis revealed that gender did not significantly impact the perceived value of the activity, instructor contribution, or personal effort across both examined activities.

Student Engagement Survey (Based on Wiggins, Eddy et. al (2017) Assessing Student Perspective of Engagement in Class Tool (ASPECT))				
Gender: Male _____		Major: Computer Science _____		
Female _____		Engineering _____		
		Other: _____		
Instructions: The following questions ask you about your experience with the More Processors are Not Always the Best activity that you completed today. Please rate how strongly you agree or disagree with each of the following statements. Circle your response				
1) Explaining the material to my group improved my understanding of it.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
2) The instructor's enthusiasm made me more interested in the More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
3) Having the material explained to me by my group members improved my understanding of the material.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
4) Group discussion during the More Processors are Not Always the Best activity contributed to my understanding of the course material.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
5) The instructor put a good deal of effort into my learning for today's class.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
6) I had fun during today's More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
7) Overall, the other members of my group made valuable contributions during the More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
8) The instructor seemed prepared for the More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
9) I would prefer to take a class that includes this group activity over one that does not include this More Processors are Not Always the Best group activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
10) I am confident in my understanding of the material presented during today's More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
11) I made a valuable contribution to my group today.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
12) The instructor and TAs were available to answer questions during the More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
13) The More Processors are Not Always the Best activity increased my understanding of the course material.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
14) I was focused during today's More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
15) The More Processors are Not Always the Best activity stimulated my interest in the course material.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree
16) I worked hard during today's More Processors are Not Always the Best activity.				
1-strongly disagree	2-somewhat disagree	3-neither agree or disagree	4-somewhat agree	5-strongly agree

Figure 1.3: Modified ASPECT survey used for the Penny Sorting unplugged activity

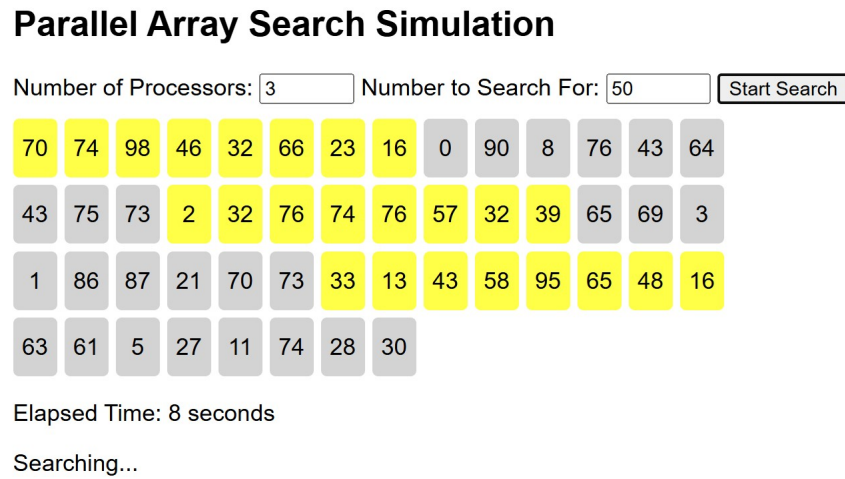
Age, however, showed a significant effect on instructor contribution in the PE activity. It is important to note that this initial analysis was conducted with a single group post-test and a small sample size, which may limit the generalizability of these findings. For a more in-depth understanding of this analysis, please refer to [10]. As the research evolved, the need to incorporate a pre- and post-survey approach is recognized to assess the extent to which the activities support student learning of key PDC concepts, such as concurrency, message passing, shared memory, scheduling, load balancing, speedup, and Amdahl's law.

1.3.3 Using Animations (Sue)

Parallel Search Animation

Problem Description Searching for a target value in a large collection is one of the simplest examples of an embarrassingly parallel problem. Similar to this idea: 1.3.2. This

Figure 1.4: Parallel Search Animation



animation contrasts sequential and parallel search by allowing students to observe how dividing the search space among multiple workers can reduce execution time. See figure1.4. Students can adjust the number of workers and visualize how each worker is assigned a different portion of the dataset. The activity also introduces the idea that adding more workers does not always produce proportional speedup because of coordination overhead and workload imbalance.

The Animation can be found here: [Parallel Search Animation](#)

Prerequisites Students should understand basic loops, arrays (or lists), and linear search. No prior knowledge of parallel programming is required.

Learning Outcomes After completing this activity, students should be able to:

1. Distinguish between sequential and parallel search.
2. Explain how search tasks can be partitioned among multiple workers.
3. Describe why parallel search can reduce execution time.
4. Recognize factors that limit parallel speedup, including workload imbalance and synchronization overhead.

Parallel Linked List Animation

Problem Description Linked lists present a different set of challenges for parallel programming than arrays because elements are connected through pointers rather than contiguous memory locations. This animation demonstrates how multiple operations, including searching, inserting, and deleting nodes, can occur concurrently on a shared linked list. See Figure 1.5. Students can execute multiple operations simultaneously and observe how concurrent modifications may interfere with one another when proper synchronization is not

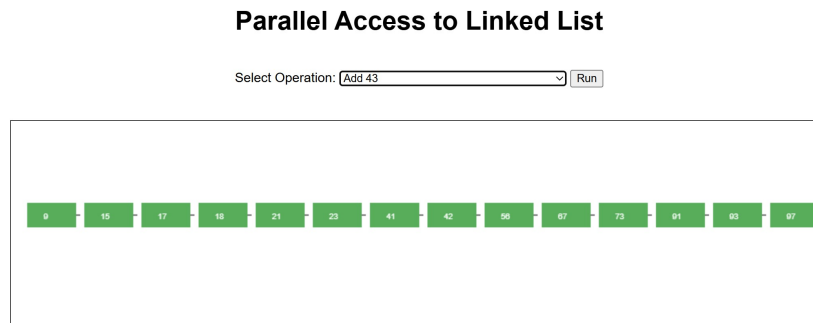


Figure 1.5: Parallel Linked List Animation

used. The animation introduces fundamental concepts such as concurrent data structures, synchronization, race conditions, and mutual exclusion intuitively and interactively.

The animation can be found here: [Parallel Linked List Animation](#)

Prerequisites Students should understand basic linked list operations, including traversal, insertion, and deletion. A basic understanding of pointers or references is helpful, although no prior knowledge of parallel programming is required.

Learning Outcomes After completing this activity, students should be able to:

1. Explain why concurrent operations on linked lists require synchronization.
2. Distinguish between read-only operations (search) and modifying operations (insert and delete).
3. Recognize how race conditions can arise when multiple threads access a shared linked list simultaneously.
4. Describe the role of mutual exclusion in maintaining data consistency during concurrent execution.

Flag Coloring Animation

Problem Description The Flag Coloring animation introduces task decomposition through an interactive coloring activity inspired by the classic map-coloring problem. Similar to the unplugged activity described in Section 1.3.1, students are asked to assign different workers to color independent regions of a flag while ensuring that adjacent regions do not share the same color. See Figure 1.6. As students assign work to multiple workers, they observe that many regions can be colored concurrently, whereas neighboring regions introduce dependencies that require coordination. The activity provides an intuitive introduction to task decomposition, concurrency, synchronization, and dependency analysis without requiring any programming experience.

The animation can be found here: [flag coloring animation](#)

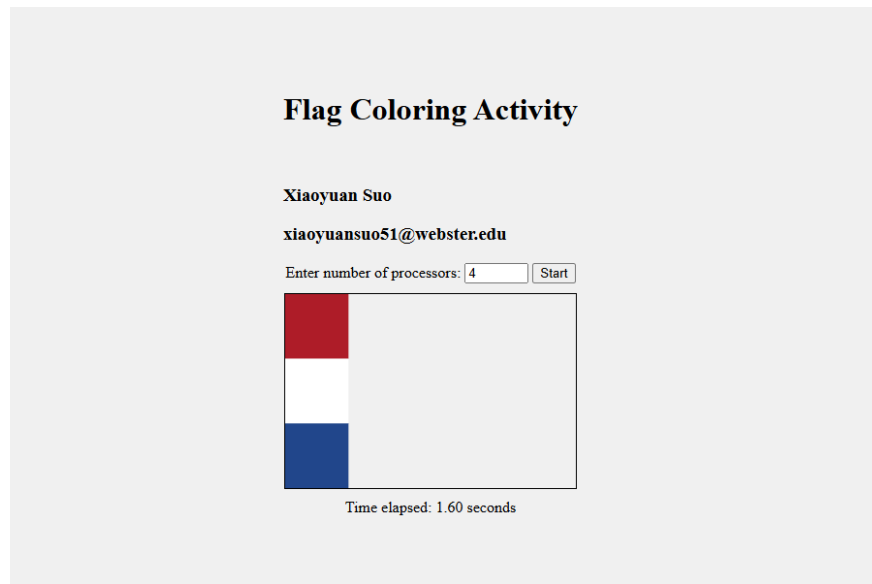


Figure 1.6: Flag Coloring Animation

Prerequisites No programming experience is required. The activity is suitable for CS0, CS1, CS2, or non-computing audiences. Students only need a basic understanding of following rules and solving simple logical puzzles.

Learning Outcomes After completing this activity, students should be able to:

1. Explain the concept of task decomposition by dividing a larger problem into smaller independent tasks.
2. Identify tasks that can be executed concurrently and tasks that require coordination because of dependencies.
3. Recognize that not all work can be performed in parallel due to shared constraints.
4. Relate graphical task decomposition to the design of parallel algorithms and workload distribution.

Zombie Attack Animation

Problem Description The Zombie Attack animation introduces parallel computing concepts through a simulation of a zombie outbreak spreading across a two-dimensional grid. Similar to the programming activity described in Section ??, each cell in the grid represents an individual who may be healthy or infected. During each simulation step, infected individuals may spread the infection to their neighboring cells. See Figure 1.7. Students can divide the grid among multiple workers and observe how different regions are processed concurrently while neighboring partitions must exchange information at their boundaries. The activity provides an intuitive introduction to domain decomposition, parallel simulation, synchronization, and communication between neighboring tasks.

The animation can be found here: [Zombie Attack animation](#)

Zombie Outbreak Simulation

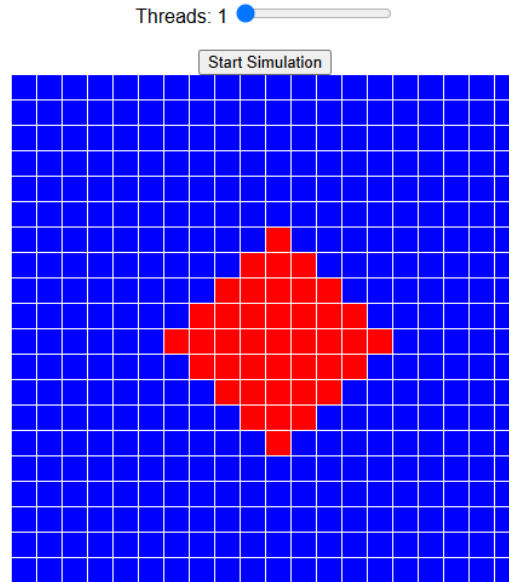


Figure 1.7: Zombie Attack Animation

Prerequisites Students should understand basic loops, conditional statements, and two-dimensional arrays (or matrices). No prior knowledge of parallel programming is required.

Learning Outcomes After completing this activity, students should be able to:

1. Explain how a two-dimensional problem can be partitioned into smaller regions for parallel execution.
2. Describe the concept of domain decomposition and its role in parallel simulations.
3. Recognize that neighboring partitions must communicate to correctly update boundary cells.
4. Explain how synchronization ensures that all workers complete one simulation step before advancing to the next.
5. Relate the simulation to real-world parallel applications such as disease modeling, wildfire spread, weather forecasting, and image processing.

Experience and Teaching Tips for All the Animations

More discussions can be found in my previous conference publication: [11]

We found that animations were an effective way to introduce abstract PDC concepts before students encountered programming implementations. Compared with traditional unplugged activities, animations required substantially less classroom time while still promoting

How helpful was the activity in learning following concepts?

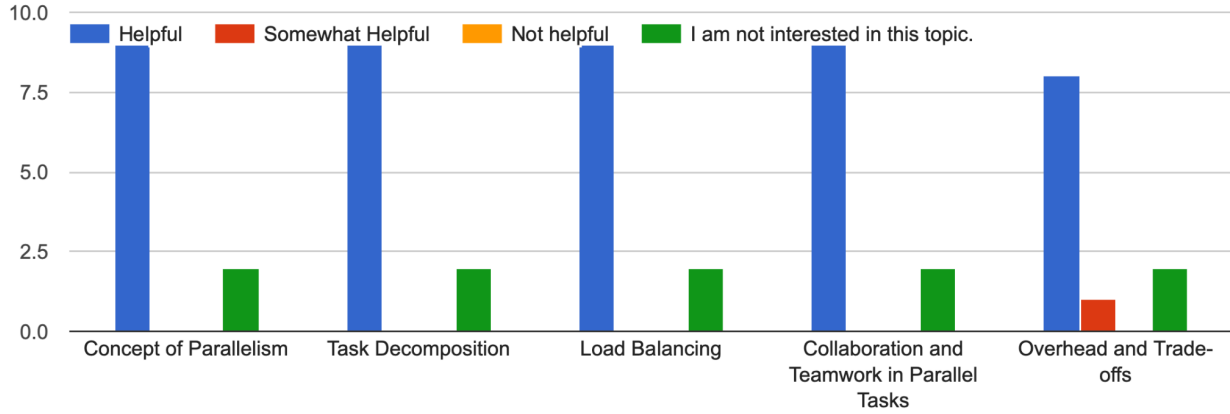


Figure 1.8: Responses from participants

discussion and conceptual understanding. Students generally found the visualizations engaging because they could immediately observe the effects of changing parameters, such as the number of workers, workload distribution, or decomposition strategy.

Animations worked particularly well when integrated with short instructor-led discussions rather than being presented as standalone demonstrations. Instructors were encouraged to pause the animation periodically and ask students to predict the outcome before continuing the simulation. These prediction activities often revealed common misconceptions about parallel execution and provided opportunities for immediate feedback.

Although the animations are designed to be self-contained, they are most effective when followed by a simple programming exercise or classroom discussion that connects the visualization to actual implementations. Instructors should also emphasize that animations intentionally simplify many aspects of parallel execution in order to highlight the underlying concepts rather than the implementation details.

Data and Analysis

Survey data were analyzed using descriptive statistics to assess changes in students' perceptions and understanding of parallel computing concepts. Specifically, Likert-scale responses from the post-activity survey were summarized to gauge the perceived effectiveness and engagement of the unplugged activity.

Key findings from the post-activity survey include:

- 90% of participants agreed that the activity reinforced their learning and helped them understand how parallel computing is applied in real-world contexts.
- 81% of participants agreed that the flag coloring activity increased their interest in parallel computing.

- 81% of participants indicated a preference for classes that incorporate group activities over those that do not.
- 90% of participants agreed that the activity was helpful in understanding key concepts such as parallelism, task decomposition, load balancing, collaboration and teamwork in parallel tasks, and overhead and tradeoffs.

Reflection responses were thematically coded to identify recurring themes related to conceptual understanding, engagement, and perceived value of the activity. The submissions to the Python parallel programming exercises were evaluated on the basis of precision, correct application of the parallel constructs, and clarity of implementation. Classroom observation notes provided additional qualitative evidence of student participation and collaborative interactions during the activities.

I feel prepared to apply parallel computing concepts to new problems beyond this activity.

11 responses

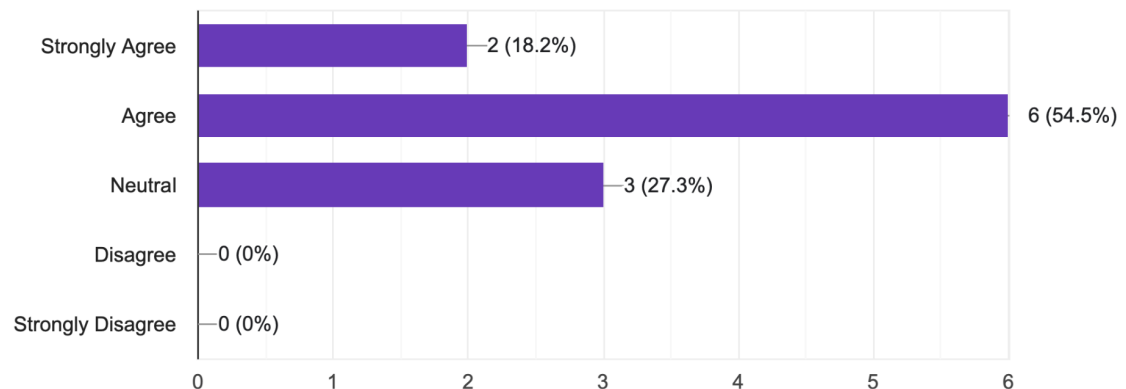


Figure 1.9: The students were then asked if they felt prepared to apply parallel computing concepts to new problems beyond this activity. 18.2 % strongly agreed with it. 54.5% agreed and 27.3 % were neutral.

Example Implementations

The following institutions are using a variation of this assignment and you can find their details in their specific chapters:

- Webster CS1- Section ??

1.4 New Plugged-in Activities

1.4.1 Programming Flag Maker (Used by Knox and USI in CS1)

Problem Description, Prerequisites, and Learning Outcomes

The Flagmaker assignment teaches nested loops by asking students to write code that draws flags on a 2D grid. For example, the code in Figure 1.10 demonstrates one way to draw the Lithuanian flag, while Figure 1.11 shows the visualization generated by Flagmaker.

```
for (int r=0; r<grid.getNumRows()/3; r++)
{
    for (int c=0; c<grid.getNumCols(); c++)
    {
        grid.setColor(r, c, Color.YELLOW);
        grid.setColor(r + R/3, c, Color.GREEN);
        grid.setColor(r + 2 * R/3, c, Color.RED);
    }
}
```

Figure 1.10: Example C program displayed as verbatim text.

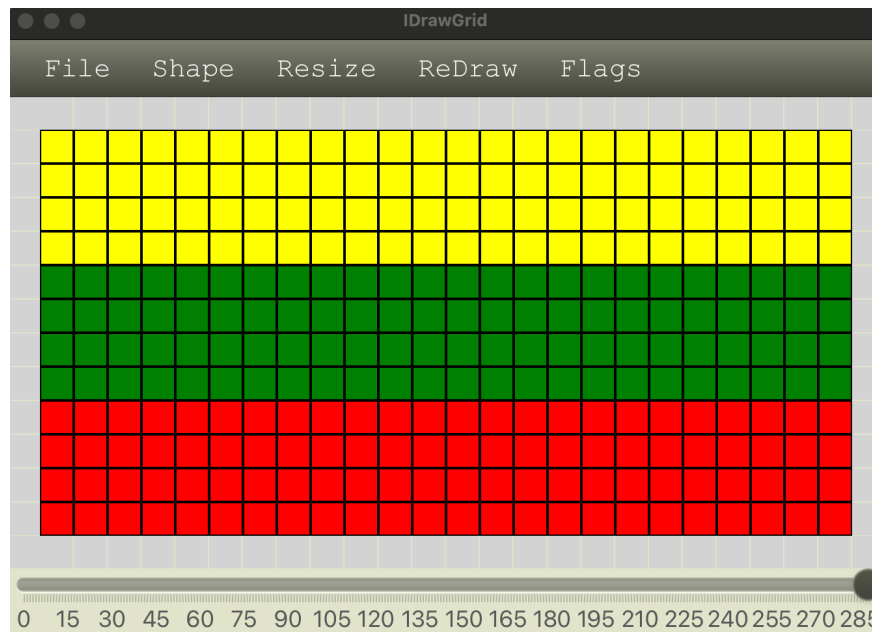


Figure 1.11: The Lithuanian flag, as rendered by the Flagmaker assignment.

Algorithms

Implementation Details

The Flagmaker assignment is based on an assignment from the mid-2000s originally created by Fawzi Emad, now a senior lecturer at the University of Maryland at College Park [4]. The original assignment, written in Java, used Java Swing. The current version uses JavaFX, which has pros and cons.

In general, demand for desktop GUI applications written in languages like Java is shrinking because many desktop apps have been replaced by control panels that connect to cloud-based SaaS (Software as a Service) backends. For those applications, HTML, CSS, and JavaScript are the dominant medium for structuring and styling text-heavy dashboard applications with standard input/output widgets (radio buttons, text inputs, checkboxes, etc). There are now tech stacks that build desktop applications with raw HTML, CSS, and JavaScript, such as Electron, which bundles in 150-200 Mb of Chromium and Node.js to handle rendering, or Tauri, which uses the platform's native browser for rendering. Thus, while we use JavaFX for the implementation, the students do not see any of these details, and we keep as much of this hidden from students as possible.

Styling desktop apps in Swing is incredibly tedious because Swing has no equivalent to CSS and so styling must all be done programmatically; meanwhile, JavaFX can be styled with a flavor of CSS, but it differs from regular CSS in enough ways that it's unclear that it is worth learning outside of the specific use cases where a Java desktop GUI application is required.

- **Instructional Material (Lectures, Videos, Assignments, Tests)**

The Flagmaker assignment asks students to implement a variety of different flags using nested for loops. The assignment should provide students with a picture of the actual flag, and a picture of what we expect their code to generate. This is necessary, as we want students to simplify complex flags in the same way each time. For example, in Figure 1.12, we show the actual Japanese flag, as well as how we expect students to approximate the flag of Japan.

The Flagmaker code includes an "autograder" feature that runs each student's flag code for a variety of valid sizes for that flag, and compares these outputs to the correct output. We don't ship any code to produce the correct flag output programmatically with the starter code for the students; instead we include the solutions as text files that the autograder reads into an internal format using library code that is hidden from the students. This is not a perfect solution, but so far has worked well enough in practice.

For example, we include a text file such as what is shown in Figure 1.13; The Flagmaker code reads this file into an internal format, then runs student code for the same flag size to produce the same internal format, and then the autograder diffs them and includes it in a report for faculty or TAs to read in order to speed up grading.

- **How-To Guides**

Our main how-to guide is a YouTube video that explains how the Flagmaker system works.

Experience and Teaching Tips

The most important tip is to change the flags every term! If we don't change the flags, previous versions of the assignment will be submitted. Thus far, we have found that desperate students will submit a copy of an entire assignment from a previous term, but are less likely to get access to copies of a previous version of the assignment, read through it, and copy only the code for specific flags. Thus, some flags can be re-used from term to term, so long as the assignment is not identical to the previous several terms. We have a large collection of flags that we mix and match each term.

We provide infrastructure to automatically generate the assignment each term, though it's typical academic code that needs some cleanup work. Please contact the authors from Knox College with any questions.

The current implementation of Flag Maker uses JavaFX, which is no longer part of regular JVM releases. Instead, JavaFX is released through OpenJDK along with each JVM release. Since JavaFX is not part of the JVM, it requires different libraries on Windows, Mac Intel, Mac with Apple silicon, and Linux. For experienced developers, this is one setting in a pom.xml or build.gradle file; for novices we avoid using Maven or Gradle whenever possible and instead use the run button in VS Code.

For Flag Maker with JavaFX, we provide copies of the different libraries, and students copy the appropriate library into the `lib` folder in VS Code. Every year, some students don't read the directions or pay attention to instructions, and then copy the wrong library, or don't copy the library, or copy the library into the wrong place, and their code doesn't compile or run. If the assignment does not run correctly, faculty or TAs should first ask: Did you copy the libraries correctly?

Another challenge is that graders should look at the outputs from the autograder, rather than simply accepting the output. Students may choose a different but related color, such as red instead of orange, but otherwise generate the flag correctly; the autograder would flag this as a large number of different pixels, when in reality this is a simple misunderstanding. Similarly, students may be "off by one" in their flag, for example by having a triangular shape that is one column too wide or too narrow. That difference matters but represents one mistake, regardless of the number of differences between the correct version and the flag the student generated. To mitigate these challenges, we suggest using rubric-based grading for this assignment (i.e. 7/7 for totally correct, 5/7 for one conceptual mistake, 3/7 for two or more conceptual mistakes but code that is reasonable, 1/7 for some code written, etc) rather than trying to measure the percentage of total grid squares that match exactly.

Can an LLM do this assignment? Yes, because LLMs can do nearly all CS-1 tightly specified CS-1 assignments. However, if a student just gives the template code to the LLM and asks it to do the assignment, it will likely generate code that is suspiciously wrong. For example, as we see in Figure 1.12, we ask students to approximate the flag of Japan as a square. Unless students prompt the LLM otherwise, it will use methods like **Math.hypot** try to draw a circle, which is wrong for the assignment and represents knowledge beyond what a typical CS-1 student would know.

Data and Analysis

We did not collect data specifically on this assignment.

1.4.2 Greenfoot (Used by Knox (CS1), USI (CS1), Caspar (CS2), MSU (CS2))

Problem Description, Prerequisites, and Learning Outcomes

Greenfoot [7] [13], developed by the team behind BlueJ [1], is a visual IDE that empowers novices to build 2D games and simulations using Java code. The key PDC concepts covered by Greenfoot are event-based programming and callbacks. A Greenfoot *scenario* is actually quite simple and elegant: Students create agents (called *actors* in Greenfoot parlance) with an `act()` method that is called every frame to update the actor. Greenfoot provides helpful utility methods for detecting if two actors are touching or overlapping, changing background images, setting the position and rotation of an actor, changing the appearance of an actor, and so on. All Greenfoot classes are in a single package, so students simply `import greenfoot.*` to access all Greenfoot utility code.

Given a few examples and their own creativity, novice students can create powerful games and simulations with only about 7 weeks of CS-1 Java knowledge. Students can make basic 2D platformers, basic RPGs, space invader style shooters, puzzles, interactive chemistry study guides, and a wide variety of other projects.

Figure 1.14 illustrates the basic Greenfoot IDE. Notice how the four actors (Phone, balloon, hedgehog, pizza) are clearly organized under the Actor, while the MyWorld world class (basically the screen where the scene takes place) is organized under World. Three of the four actors (hedgehog, balloon, pizza) can be seen in the actual scene; the Phone class does not show up in the scene intentionally to demonstrate for students that some code is required to create an instance of the Phone class and place it someplace into the scene.

In Figure 1.15, we see the `act()` method to move an actor in Greenfoot. The code is simple and is accessible to CS-1 students with about 7 weeks of Java experience.

Algorithms

The main PDC concept Greenfoot covers is event-driven programming. The `act()` method of each actor is a callback that the Greenfoot system calls each frame. By default Greenfoot calls the `act()` methods of each actor in an unspecified order, which can lead to unpredictable behaviors. Figure 1.16 shows how to use the `setActOrder` method to define a consistent order in which the `act()` methods should be invoked. Most CS-1 Greenfoot programs written by novices are unlikely to require this level of coordination to ensure correctness, but faculty can construct specific examples to demonstrate the challenge of ordering callbacks with side effects.

Another key concept Greenfoot teaches is that most modern programs do not start with a `main()` method typed into a blank screen. Instead, students must read examples and APIs to figure out how to plug their code into an existing framework that calls into their code when certain events happen. This helps teach students that, for many modern software development tasks, the *framework* is in charge, not the programmer. Game engines, even

very simple ones like Greenfoot, are a fantastic way to highlight this reality for students in a relatively intuitive and highly engaging way.

Implementation Details

Instructional Material (Lectures, Videos, Assignments, Tests)

We have assembled four complete Greenfoot examples and created instructional videos for each, in order to help faculty adopt Greenfoot in their CS-1 course.

- Hedgehog: A simple game where the hedgehog needs to eat the pizza and avoid the balloons. Focuses on handling keyboard input, and detecting when two actors bump into each other. [video](#)
- Shooting Game: Shoot birds that look suspiciously like butterflies with a cannon. Focuses on actors that spawn other actors (cannon spawns missiles), randomized movement of the birds/butterflies, actors change their sprites on collision. [video](#)
- Cards: Very simple example that creates a deck of cards and deals them across the screen. Demonstrates how to have one actor (Card) that can have many sprites (the 52 playing card images). [video](#)
- Zelda: Core pieces of 1986 NES game The Legend of Zelda. Navigate a 2D grid and change backgrounds when leaving each screen, animated sprite, spawns enemies, very basic dialog window. [video](#)

Experience and Teaching Tips

We have found two keys to successful Greenfoot assignments: First, trust the creativity of students, and second, embrace open-ended, ill-formed rather than *close-ended*, *well-formed* assignments [6].

Many introductory CS assignments are *well-formed*, *convergent*, and *close-ended* [3, 5]. CS-1 assignments are tightly specified, ambiguities are eliminated whenever possible, and students tend to converge on one of a small set of correct solutions. For example, a typical convergent, well-formed assignment is to ask students to implement linked list methods.

Well-formed assignments have many strengths: they can be carefully designed to cover specific concepts, scoped to help students manage cognitive load, aligned with unit testing to enable autograding, integrated with worked examples, pruned of ambiguity to give students a clear definition of "done", and targeted so that specific misconceptions lead to predictable outcomes that are easy for instructors to diagnose. It would be hard to imagine introductory CS education without well-formed assignments.

However, well-formed assignments serve as cognitive scaffolding for novices that prepare students to solve "real world" problems, which are often messy, ambiguous, poorly specified, unpredictable, and frequently all of the above. Well-formed assignments provide minimal opportunities for students to flex their creativity, and apply their hard-won software engineering skills to open-ended problems.

Greenfoot naturally lends itself to open-ended, divergent, ill-defined assignments, as it would be uninteresting and impractical to ask all students to implement the same game

in the same way. We typically ask students to make a game or simulation, and provide only minimal guidelines for how to do so. For example, we provide four complete worked examples, and ask students to use these as inspiration to make something new. We give guidelines about what students *must not* do (no minor tweaks or remixes of the examples, no tweaks of examples downloaded from greenfoot.org, etc), but minimal guidelines on what students *must* do. This is challenging for some students who expect very clear guidelines of what is required to receive a good grade, and so we have a couple of basic fallback options for those students, such as making a simple shooter or maze solving game. However, it has been our experience that many more students value the creative opportunities of Greenfoot far outweigh the challenges of students who seek the simplest pathway to an A on the assignment.

One concern with divergent, ill-formed, open-ended assignments is that some students do much more work than others, and that some students try to find a way to do the minimum. Our response: So what? Students often try to do the minimum, and part of teaching, advising, and mentoring is to encourage students to do more, not less. Different students do different amounts of work for a variety of reasons, such as more excitement or passion for a particular topic, work or class commitments outside of the class, whether a course is for a major or to fulfill a requirement or an elective, prior background in an area, confidence with course material, and so on. Well-formed, convergent assignments attempt to mandate a *minimum* threshold of work, but in the process they typically also set a *maximum* amount of work as well. Greenfoot assignments, at least in the way we implement them, take the opposite approach, and set minimal standards with no maximum, and then trust the students, and also our own ability to inspire, to challenge students to do more rather than less.

One word of caution: The greenfoot.org website allows students to publish their "scenarios", and then download and "remix" each other's work (it was the late 2000s and it was popular to build a repository of artifacts that could be remixed). This has resulted in an enormous library of a few classic assignments, many remixed with tiny modifications that make it difficult to find which one a panicked student downloaded and submitted right before the deadline. Because of this, we do not allow students to make versions of the **snake game**, **flappy bird**, or a couple of others that are well represented on greenfoot.org. There are just too many versions of these, and the search tools on greenfoot.org are not sophisticated enough to easily figure out which one a student submitted.

Greenfoot has been around since the late 2000s, and while it is commonly used in middle and high school, it is underutilized at the college and university level. The basic sprites included with Greenfoot are copyright free images (similar to Scratch [8]) that look appropriate for middle school, but may not present a sophisticated visual language appealing for a college audience. It is crucial not to judge the power and complexity of Greenfoot on these first impressions! It is quite easy to import custom images into a Greenfoot scenario, and we have never had complaints from college students about Greenfoot, or Scratch when we taught that years ago.

Data and Analysis

We did not collect data specifically on this assignment.

1.4.3 Plugged-In Earthquake Activity(April)

Problem Description, Prerequisites, and Learning Outcomes

The *Plugged-In Earthquake Activity*, asks students to complete a C++ program that retrieves real, live earthquake data from the United States Geological Survey (USGS), filters that data, and displays selected earthquake information to the screen. Unlike a traditional introductory programming assignment focused only on loops, functions, and basic syntax, this assignment introduces students to the broader idea of working with remote, distributed, live data through an API. Students use a USGS earthquake data feed, returned in JSON format, to see how programs can interact with real-world data sources and support socially meaningful applications such as natural disaster monitoring.

The *prerequisites* for this assignment is that students should already have experience with several foundational C++ concepts, including for loops, arrays, functions, Makefiles, JSON, cURL, and distributed data. Students are not expected to build an API or JSON parser from scratch; instead, they are given starter code, supporting libraries, and Makefile templates so they can focus on understanding how data is retrieved, parsed, filtered, and displayed.

The main *learning outcomes* are that students will be able to develop and implement an algorithmic solution for interacting with a remote service, practice working with JSON-formatted data, and compile/run a program using a Makefile. By the end of the assignment, students should understand the client-server relationship at a basic level: their computer acts as the client, sends a request to a remote USGS server through an API, receives live earthquake data, and then processes that data locally in a C++ program. A secondary learning outcome is conceptual. Students are asked to explain terms such as remote data, distributed data, API, and JSON, and to describe another real-world situation where large amounts of remote data are used to provide a public service. This documentation component reinforces the idea that computing is not only about writing isolated programs, but also about building systems that interact with live data sources in meaningful contexts.

Algorithms

The core algorithm for this assignment follows a retrieve–parse–filter–display structure. First, the program retrieves earthquake data from the USGS monthly earthquake feed. The data source is a live GeoJSON file containing detected earthquakes from the past month. Because the data is dynamic, the exact output may differ from one run to another depending on when the program is executed.

Second, the program parses the returned JSON data and accesses the features array. Each element in this array represents an earthquake event. For each earthquake, the relevant information is stored inside the properties object, including values such as magnitude, place, timestamp, and tsunami indicator.

Example Implementations

The following institutions are using a variation of this assignment and you can find their details in their specific chapters:

- TNTECH - CS1 course - Section 2.4.1
- Casper - CS1 course - Section 7.4.3

1.5 Course-wide Pre and Post Course Surveys (Chris, Neena)

Development and testing teams administered the following common pre/post survey for CS1. The survey asked students their background knowledge in computing topics, programming languages, computing concepts, and PDC concepts. Students were asked to rate their prior experience on a 7-point Likert scale:

1. No Experience At All
2. Very Little Experience
3. Slightly Experienced
4. Between Slightly/Moderately Experienced
5. Moderately Experienced
6. Very Experienced
7. Extremely Experienced

The survey consisted of the following 26 items. Adopters can modify the pre/post surveys for background knowledge in computing topics, programming languages and computing concepts expected from the students in their courses. Similarly, depending on which PDC topics they intend to introduce, they can choose parallel processing, distributed computing, and event handling.

- | | |
|---|------------------|
| 1. Computer use experience (internet searches, email, discord, discussion groups, games, word processing, spreadsheets, organizing files & folders, etc.) | 6. C# |
| 2. Programming Experience (self-initiated learning and/or formal programming course) | 7. Java |
| 3. Programming Experience compared to your classmates | 8. JavaScript |
| 4. C | 9. PHP |
| 5. C++ | 10. Python |
| | 11. Scratch |
| | 12. Visual Basic |
| | 13. Other (any) |

- | | |
|---|--|
| 14. Data/Variable Types (integers, doubles, char etc.) | 23. Creating objects |
| 15. Arithmetic Expressions and Operators | 24. Parallel processing: Computing where the operations within a program are being executed simultaneously (i.e. running on different cores of the processor at the same time) |
| 16. Branching (if/else, relational and logical operators, relational expressions) | |
| 17. Loops (while, for, etc.) | 25. Distributed computing: Computing with a program that runs using data from a different computer (i.e. an on-line service) |
| 18. Calling functions/methods | |
| 19. Arrays | 26. Event handling: Having a function that responds to an independent action (i.e. a mouse click) |
| 20. Writing static functions/methods | |
| 21. Writing functions/methods | |
| 22. Writing classes | |

1.6 Conclusion

The activities in this chapter show that parallel and distributed computing can be introduced meaningfully in CS1. Across the institutional exemplars, the most successful approaches connect PDC ideas to familiar CS1 topics such as loops, arrays, searching, sorting, functions, objects, APIs, graphical interaction, and event handling. Unplugged activities, animations, visualizations, and short programming assignments help students first build intuition about workers, task decomposition, speedup, load imbalance, contention, synchronization, communication, remote data, and event-driven control before encountering more detailed implementation issues. A central lesson is that PDC is most effective in CS1 when it reframes and enriches existing course goals rather than appearing as disconnected additional content.

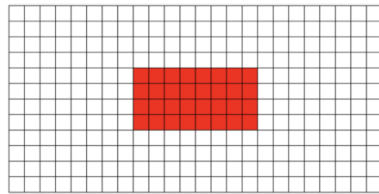
For instructors wishing to adopt, adapt, or extend these materials, the best starting point is often a small, feasible activity followed by a careful debrief that helps students name and interpret what they observed. Local constraints matter: class size, programming language, lab structure, student background, available teaching-assistant support, software setup, and instructional time all influence which activity should be chosen and how it should be adapted. The institutional chapters, appendices, and repositories provide implementation details, slides, handouts, starter code, rubrics, surveys, timing guidance, and classroom evidence to support such adaptation. These instructional materials, assessment instruments, and evidence summaries are expected to be periodically curated and updated as additional adopters use, revise, and extend the activities in new CS1 contexts.

Japan



The flag of Japan has a white background with a red circle in the middle. Drawing circles is very difficult, so we will approximate the circle with a rectangle. Your rectangular red dot should be in the middle third of the flag. The height and width of the flag must be a multiple of 3.

Size 12



Size 24

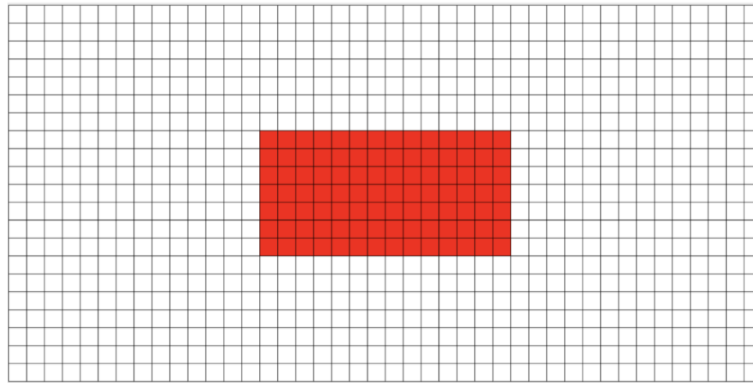


Figure 1.12: The assignment piece for the Japanese flag.

```

YYYYYYYYYYYY
YYYYYYYYYYYY
GGGGGGGGGGGG
GGGGGGGGGGGG
RRRRRRRRRRRR
RRRRRRRRRRRR

```

Figure 1.13: Solution as text for the Lithuanian flag.

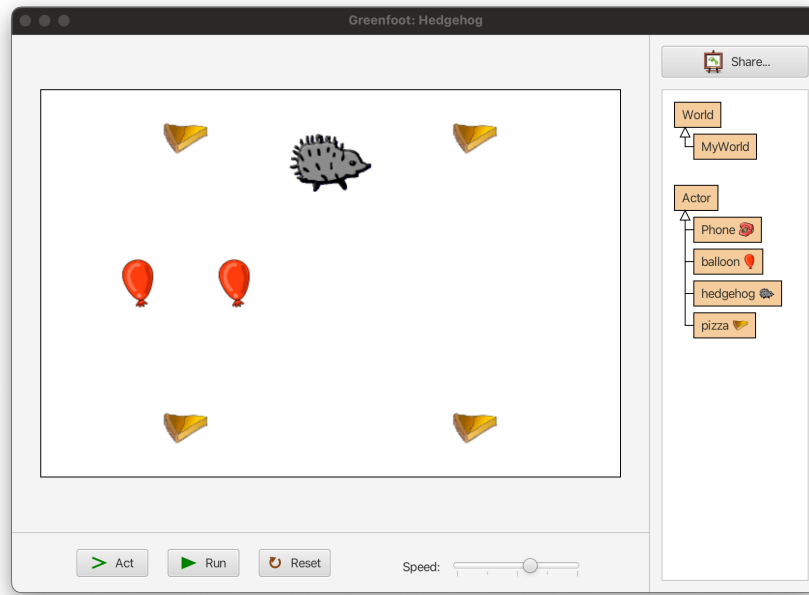


Figure 1.14: The Hedgehog example in Greenfoot. Notice how Actors are clearly organized in the IDE

```
int speed = 3;
public void act() {
    if (Greenfoot.isKeyDown("right")) {
        setLocation(getX()+speed, getY() );
    }
    if (Greenfoot.isKeyDown("Left")) {
        setLocation(getX()-speed, getY() );
    }
    if (Greenfoot.isKeyDown("Up")) {
        setLocation(getX(), getY()-speed );
    }
    if (Greenfoot.isKeyDown("Down")) {
        setLocation(getX(), getY()+speed );
    }
}
```

Figure 1.15: Example of how to move an actor in Greenfoot. Note the simplicity and elegance of the code.

```
public class MyWorld extends World {  
    public MyWorld() {  
        super(600, 400, 1);  
        // Set execution order by Class  
        setActOrder(Player.class, Enemy.class, Scoreboard.class);  
    }  
}
```

Figure 1.16: Example of `setActOrder()` method showing how to define a specific order for Greenfoot to call the `act()` methods.

Bibliography

- [1] Neil C. C. Brown. Bluej 5: Still going strong. In Proceedings of the 51st ACM Technical Symposium on Computer Science Education, SIGCSE '20, page 1420, New York, NY, USA, 2020. Association for Computing Machinery.
- [2] April R. Crockett, Gerald C. Gannod, Xiaoyuan Suo, Chris Bourke, Mary L. Smith, Srishti Srivastava, David P. Bunde, Jaime Spacco, Jiayin Wang, Michelle Zhu, Neena Thota, Charles C. Weems, R. Vaidyanathan, Alan Sussman, and Sushil K. Prasad. Making room for parallel and distributed computing in CS1: approaches, tradeoffs, and faculty effort. In 2026 Frontiers in Education (FIE), pages –. IEEE, 2026.
- [3] Educational Content Team. Convergent thinking. Cogn-IQ Encyclopedia, 2026.
- [4] Fawzi Emad. Flag coloring programming assignment. Personal communication to J. Spacco.
- [5] Joy Paul Guilford. The structure of intellect. Psychological Bulletin, 53(4):267–293, 1956.
- [6] David H. Jonassen. Toward a design theory of problem solving. Educational Technology Research and Development, 48(4):63–85, 2000.
- [7] Michael Kölling. The greenfoot programming environment. ACM Trans. Comput. Educ., 10(4), November 2010.
- [8] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. Scratch: programming for all. Commun. ACM, 52(11):60–67, November 2009.
- [9] M. Smith, S. Srivastava, D.P. Bunde, A. Crockett, M. Gerten, P. Maher, J. Spacco, X. Suo, J. Wang, and M. Zhu. A visual unplugged activity to introduce PDC. In Proc. Workshop on Education for High-Performance Computing (EduHPC), 2025.
- [10] Srishti Srivastava, Mary Smith, Amrita Ghimire, and Shaun Gao. Assessing the integration of parallel and distributed computing in early undergraduate computer science curriculum using unplugged activities. In 2019 IEEE/ACM Workshop on Education for High-Performance Computing (EduHPC), pages 17–24. IEEE, 2019.

- [11] Xiaoyuan Suo and Timila Dangol. Engaging First and Second Year Undergraduates with Parallel and Distributed Computing: Lessons from Unplugged and Game-Based Activities, pages 104–114. 06 2026.
- [12] Wikipedia user Zscout370. Flag of mauritius. https://en.wikipedia.org/wiki/Flag_of_Mauritius#/media/File:Flag_of_Mauritius.svg, viewed Jun 19, 2026.
- [13] Tamar Vilner, Ela Zur, and Shay Tavor. Integrating Greenfoot into CS1: a case study. In Proceedings of the 16th Annual Joint Conference on Innovation and Technology in Computer Science Education, ITiCSE '11, page 350, New York, NY, USA, 2011. Association for Computing Machinery.
- [14] Benjamin L. Wiggins, Sarah L. Eddy, Lauren Wener-Fligner, Katherine Freisem, Daniel Z. Grunspan, Elli J. Theobald, Jessica Timbrook, and Alison J. Crowe. ASPECT: A Survey to Assess Student Perspective of Engagement in an Active-Learning Classroom. CBE—Life Sciences Education, 16(2):ar32, Summer 2017.